Assignment-18.3

Name : Boddu Tejas Kumar
Htno :2303a51489
Batch-08

Lab 18 – API Integration: Connecting to External Services with Error Handling

Task 1 – Public Transport Route Fare API

Prompt:
Generate a Python program using the requests library to fetch public transport fare between two stations using an API. Include input validation and error handling for invalid inputs, API failure, and timeouts.

```python
import requests

API_KEY = "kjndskjsdnascascjsanc,"

def get_fare(source, destination):
    if not source or not destination:
        print("Invalid input: Source/Destination required")
        return

    url = "https://api.distancematrix.ai/maps/api/distancematrix/json"

    params = {
        "origins": source,
        "destinations": destination,
        "key": API_KEY
    }

    try:
        response = requests.get(url, params=params, timeout=5)

        # If API fails → fallback
        if response.status_code != 200:
            print("API failed, using fallback calculation...")
            fallback_fare(source, destination)
            return

        data = response.json()

        if data.get("status") != "OK":
            print("API error, using fallback...")
            fallback_fare(source, destination)
            return
```

```python
        element = data["rows"][0]["elements"][0]

        if element.get("status") != "OK":
            print("Route not found, using fallback...")
            fallback_fare(source, destination)
            return

        distance_text = element["distance"]["text"]
        distance_km = float(distance_text.split()[0])

        fare = 10 + (distance_km * 2)

        print("\n--- Fare Details (API) ---")
        print("Source:", source)
        print("Destination:", destination)
        print("Distance:", distance_text)
        print(f"Fare: ₹{fare:.2f}")

    except requests.exceptions.RequestException:
        print("Network error, using fallback...")
        fallback_fare(source, destination)


def fallback_fare(source, destination):
    # simple approximate distance
    approx_distance = 500   # fixed for demo
    fare = 10 + (approx_distance * 2)

    print("\n--- Fare Details (Fallback) ---")
    print("Source:", source)
    print("Destination:", destination)
    print("Distance: Approx 500 km")
    print(f"Fare: ₹{fare:.2f}")
```

```python
# Example
get_fare("Hyderabad", "Bangalore")
```

```
API error, using fallback...

--- Fare Details (Fallback) ---
Source: Hyderabad
Destination: Bangalore
Distance: Approx 500 km
Fare: ₹1010.00
```

Task 2 – Currency Exchange Rates

Prompt:
Write a Python script to convert INR to USD, EUR, and GBP using a currency exchange API. Include proper error handling for invalid input, API failure, and display results clearly.

```python
import requests

def convert_currency(amount):
    if amount <= 0:
        print("Invalid amount")
        return

    try:
        url = "https://api.exchangerate-api.com/v4/latest/INR"
        response = requests.get(url, timeout=5)

        data = response.json()

        usd = amount * data['rates']['USD']
        eur = amount * data['rates']['EUR']
        gbp = amount * data['rates']['GBP']

        print("\n--- Currency Conversion ---")
        print(f"INR {amount} =")
        print(f"USD: {usd:.2f}")
        print(f"EUR: {eur:.2f}")
        print(f"GBP: {gbp:.2f}")

    except requests.exceptions.RequestException:
        print("Error: API request failed")

# Example
convert_currency(1000)
```

```
...
    --- Currency Conversion ---
    INR 1000 =
    USD: 10.70
    EUR: 9.32
    GBP: 8.13
```

Task 3 – GitHub Repository Info Fetcher

Prompt:

Create a Python function that uses the GitHub API to fetch repository details like name, description, stars, forks, and open issues. Include error handling for 404 errors, rate limits, and invalid input.

```python
import requests

def fetch_github_repo(owner, repo):
    url = f"https://api.github.com/repos/{owner}/{repo}"

    try:
        response = requests.get(url)

        if response.status_code == 404:
            print("Error: Repository not found. Check the owner or repo name.")
            return
        elif response.status_code == 403:
            print("Error: Rate limit exceeded. Try again later.")
            return

        response.raise_for_status()
        data = response.json()

        print(f"\n--- Repository Details ---")
        print(f"Name: {data['name']}")
        print(f"Description: {data['description']}")
        print(f"Stars: {data['stargazers_count']}")
        print(f"Forks: {data['forks_count']}")
        print(f"Open Issues: {data['open_issues_count']}")

    except requests.exceptions.RequestException as e:
        print(f"Network error: {e}")

owner_input = input("Enter GitHub Username: ")
repo_input = input("Enter Repository Name: ")
fetch_github_repo(owner_input, repo_input)
```

```
Enter GitHub Username: Eshwarmanupati
Enter Repository Name: AI-Assisted-Coding

--- Repository Details ---
Name: AI-Assisted-Coding
Description: None
Stars: 0
Forks: 0
Open Issues: 0
```

Task 4 – Real-Time Application: News Headlines Aggregator

Prompt:

Generate a Python program that fetches top 5 news headlines using a News API based on category input. Include error handling for invalid category, API key issues, and implement retry mechanism for failed requests.

```python
import requests
import time

API_KEY = "de1f879075744749be6a429785dc6177"

def get_news(category):
    url = f"https://newsapi.org/v2/top-headlines?category={category}&apiKey={API_KEY}"

    for attempt in range(2):  # retry mechanism
        try:
            response = requests.get(url, timeout=5)

            if response.status_code == 200:
                data = response.json()
                articles = data.get("articles", [])[:5]

                print("\n--- Top Headlines ---")
                for i, article in enumerate(articles, 1):
                    print(f"{i}. {article['title']}")
                return

            elif response.status_code == 401:
                print("Invalid API Key")
                return

            else:
                print("Retrying...")
                time.sleep(2)

        except requests.exceptions.RequestException:
            print("Network error, retrying...")
            time.sleep(2)
```


```python
                return

            elif response.status_code == 401:
                print("Invalid API Key")
                return

            else:
                print("Retrying...")
                time.sleep(2)

        except requests.exceptions.RequestException:
            print("Network error, retrying...")
            time.sleep(2)
```


```python
    print("Failed after retries")

# Example
get_news("technology")
```

```
...
--- Top Headlines ---
1. US Patent Office rejects Nintendo's 'summon subcharacter and let it fight' patent - gamesindustry.bi
2. Will your Samsung Galaxy device get stable One UI 8.5 update in April 2026 - Sammy Fans
3. Unlock rewards for starting GameChat-only with Nintendo Switch Online - Nintendo
4. Steam On Linux Use Skyrocketed In March - More Than Double The macOS Gaming Marketshare - Phoronix
5. Google AI Pro now comes with 5 TB of storage, no price increase - 9to5Google
```

Task 5 -COVID-19 Statistics API Integration

Prompt:

Write a Python program using requests to fetch COVID-19 statistics for a given country. Include total cases, deaths, recovered, and active cases. Handle errors like invalid country name, API rate limit, and network failures with retry logic.

```python
import requests
import time

def covid_stats(country):
    url = f"https://disease.sh/v3/covid-19/countries/{country}"

    for attempt in range(3):
        try:
            response = requests.get(url, timeout=5)

            if response.status_code == 200:
                data = response.json()

                print("\n--- COVID Stats ---")
                print("Country:", country)
                print("Total Cases:", data['cases'])
                print("Deaths:", data['deaths'])
                print("Recovered:", data['recovered'])
                print("Active:", data['active'])
                return

            elif response.status_code == 404:
                print("Invalid country name")
                return

            elif response.status_code == 429:
                print("Rate limit hit. Retrying...")
                time.sleep(2)

            else:
                print("Error:", response.status_code)
```

```python
        except requests.exceptions.RequestException:
            print("Network error, retrying...")
            time.sleep(2)

    print("Failed after retries")

# Example
covid_stats("India")
```

```
...
--- COVID Stats ---
Country: India
Total Cases: 45035393
Deaths: 533570
Recovered: 0
Active: 44501823
```